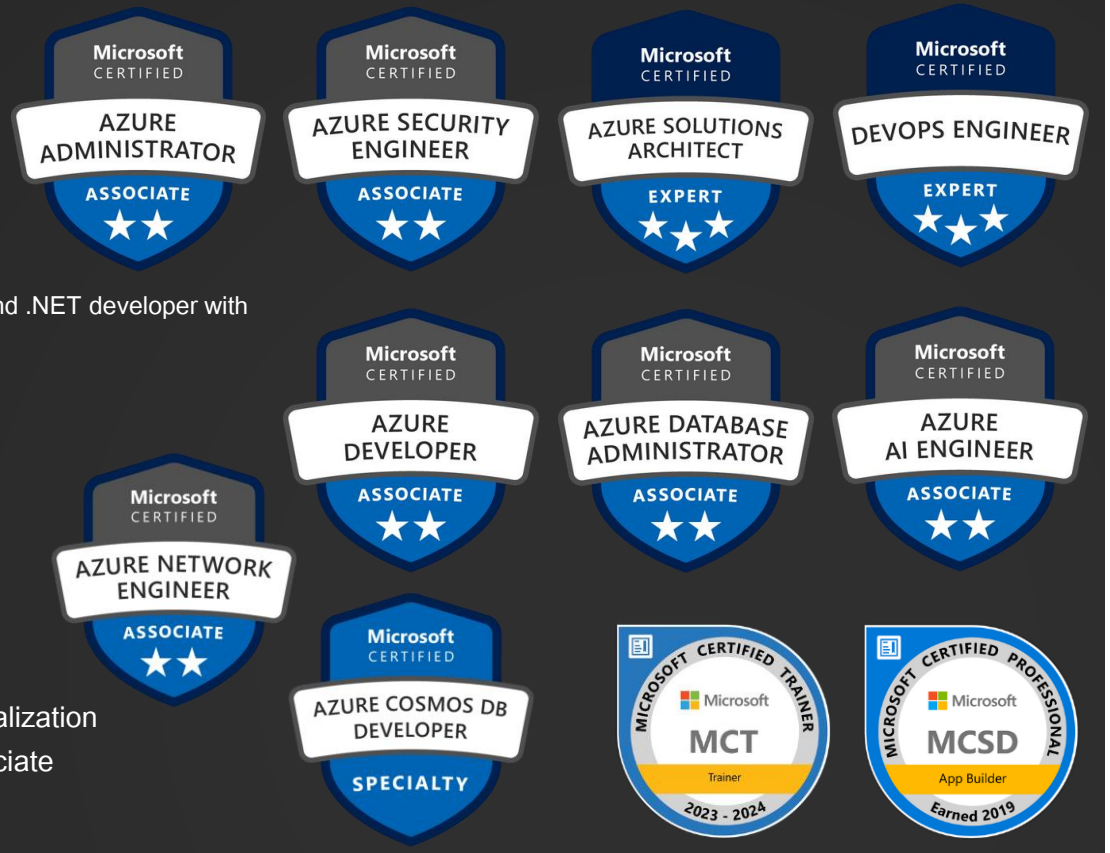


Creating Domain Invariants

Hardening an anemic Order, one technique at a time

Who am I?



Brian is an experienced speaker, author, trainer, and .NET developer with

- MCT (x6) / MVP (x5)
- MCSD: App Builder
- Azure Fundamentals
- Azure Data/SCI/AI Fundamentals
- Azure Administrator Associate
- Azure Security Engineer Associate
- Azure Developer Associate
- Azure AI Engineer Associate
- Azure Cosmos DB Developer Specialization
- Azure Database Administrator Associate
- Azure Solutions Architect Expert
- Azure DevOps Engineer Expert

Brian has a masters of science degree in computer information systems, and a bachelor of science degree in computer science. Additionally, Brian has over ten years of experience instructing college courses online in SQL databases, C#/VB .Net programming, Java programming, and Microsoft Office.

Brian has also created many online video courses, available on various platforms. Recently, Brian published the second edition of his first book: Practical Entity Framework Core 6 with Apress and an exam companion for the AZ-204 exam.

Brian runs a company named Major Guidance Solutions, which does contract and custom training and curriculum development, as well as Azure consulting and upskilling. If you are interested in becoming Azure certified, make sure to connect with me.

@blgorman || https://www.linkedin/in/brianlgorman

MVP

Most Valuable Professionals



Brian Gorman

United States

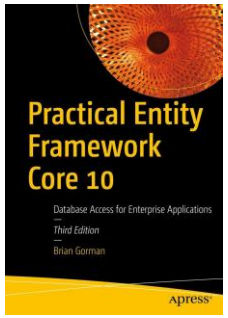
Major Guidance Solutions, L. L. C.

He/Him/His

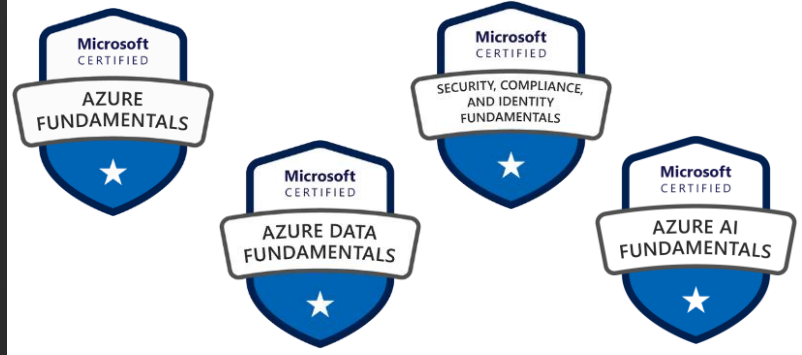
5 years in the program

Award Category

Microsoft Azure







Agenda



- The anemic Order
- The desired Order invariants
- Seven techniques to make the invariant model
- The anemic Order next to the domain Order



The Anemic Order and Its Invariants

What the Order allows today, and the rules the Order should keep

The Anemic Order

OrderModels/Anemic/Order.cs

```
public class Order
{
    public int Id { get; set; }
    public string Status { get; set; } = string.Empty;
    public decimal Total { get; set; }
    public DateTime OrderDate { get; set; }
    public string CustomerName { get; set; } = string.Empty;
    public List<OrderLine> Lines { get; set; } = new();
}
```

The Desired Order Invariants

1. Every order has a customer and at least one line.
2. The order total equals the sum of its lines.
3. Every quantity is positive, and each product appears only once.
4. Every line on an order uses the same currency.
5. An order moves from Placed to Processing to Confirmed to Shipped.
6. A cancelled order cannot be shipped.



Private Constructor and Order.Place

Every new order goes through one guarded factory method

Public Constructor: Default Values

OrderModels/Anemic

```
var order = new Order();

Customer:     ''
Status:       ''
Order date:   1/1/0001
Lines:        0
Total:        0
```

The public constructor lets any caller create a new order without any enforced business rules

Object Initializer has no constraints

OrderModels/Anemic

```
var invalidOrder = new Order { Status = "Banana", Total = -99.99m };

// Saved and reloaded through EF Core:
// Customer: ''   Status: 'Banana'   Order date: 1/1/0001
// Lines: 0       Total: -99.99
```

Any caller can build any order, and EF Core saves it.

The Order Constructor Becomes Private

OrderModels/Domain/Order.cs, after Step 1

```
private Order() { }
```

EF Core rebuilds a reloaded order through the private constructor.

Creating Orders with Order.Place

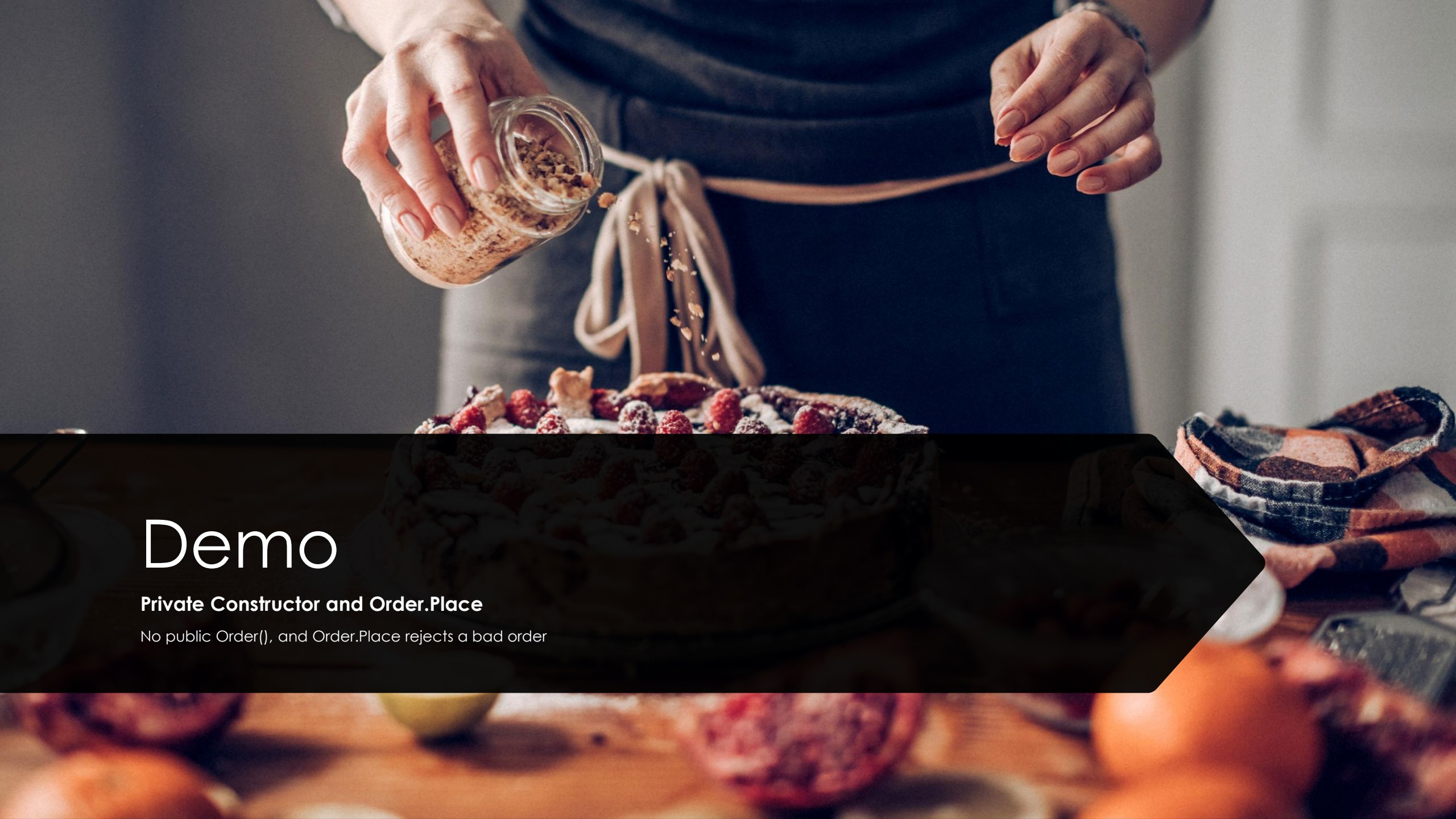
OrderModels/Domain/Order.cs, after Step 1

```
public static Order Place(string customerName,
    IEnumerable<(int productId, int quantity, Money unitPrice)> lines)
{
    var order = new Order
    {
        CustomerName = customerName,
        OrderDate = DateTime.UtcNow,
        Status = OrderStatus.Placed
    };
    foreach (var (productId, quantity, unitPrice) in lines)
    {
        order.Lines.Add(new OrderLine(productId, quantity, unitPrice));
    }
    order.RecalculateTotal();
    return order;
}
```

Guard Clauses in Order.Place

OrderModels/Domain/Order.cs, after Step 1

```
// Order.Place
if (string.IsNullOrWhiteSpace(customerName))
    throw new ArgumentException("An order needs a customer name.", nameof(customerName));
if (!lines.Any())
    throw new ArgumentException("An order needs at least one line.", nameof(lines));
```



Demo

Private Constructor and Order.Place

No public Order(), and Order.Place rejects a bad order



Private Setters

A placed order changes only through code inside the Order class

Private Setters: The Problem

OrderModels/Anemic

```
public decimal Total { get; set; }
public DateTime OrderDate { get; set; }
public string CustomerName { get; set; } = string.Empty;

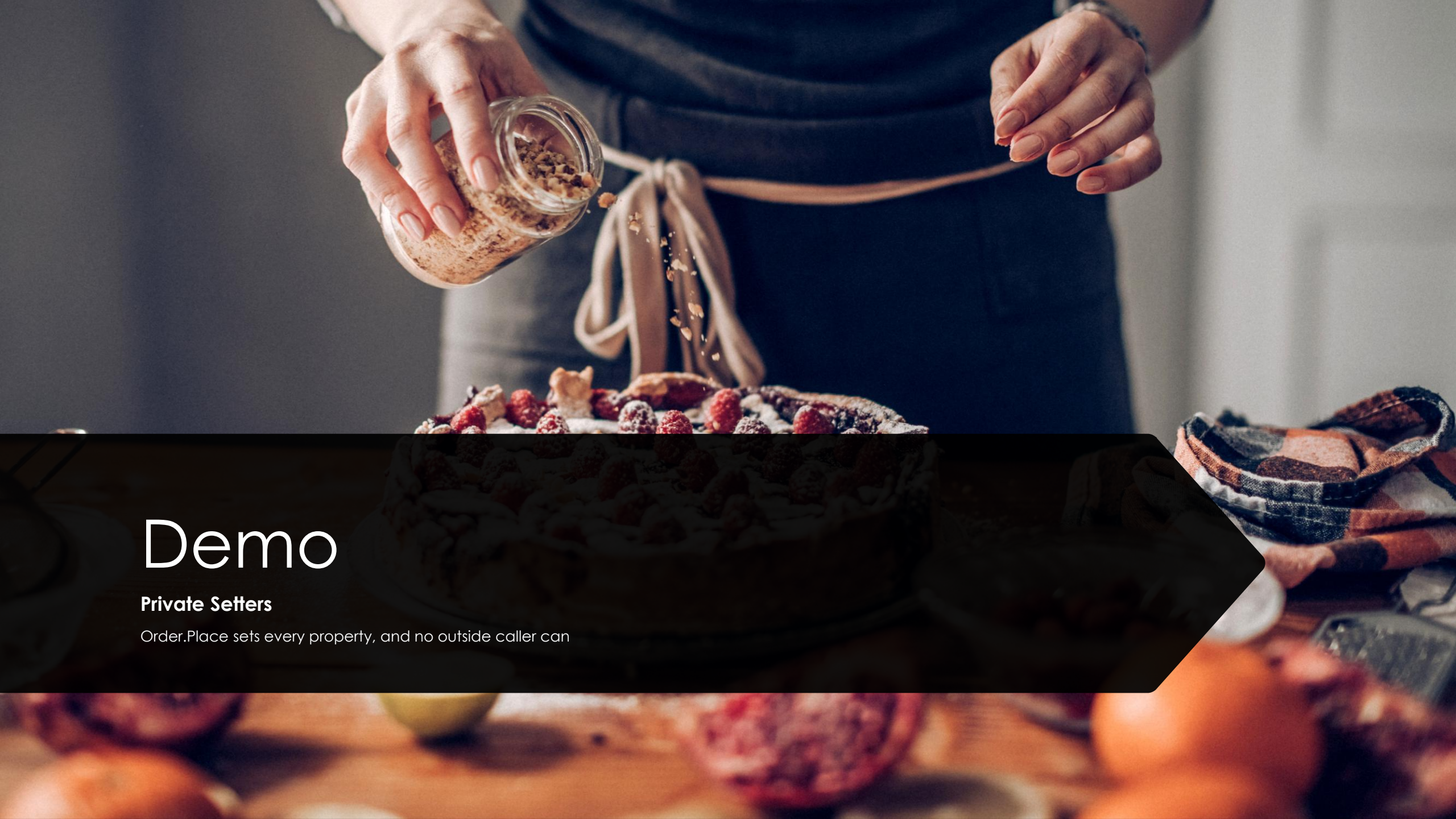
order.OrderDate = DateTime.Now.AddDays(-30);
order.CustomerName = "Someone Else";
order.Total = 0.01m;
```

Every setter is public, so any caller can change a placed order.

Private Setters

OrderModels/Domain/Order.cs, after Step 2

```
public int Id { get; private set; }
public OrderStatus Status { get; private set; }
public decimal Total { get; private set; }
public DateTime OrderDate { get; private set; }
public string CustomerName { get; private set; } = string.Empty;
```



Demo

Private Setters

Order.Place sets every property, and no outside caller can



Domain Methods Control Status

Status moves only through methods that guard each transition

Domain Methods: The Problem

OrderModels/Anemic

```
shippedOrder.Status = "Shipped";     // a Placed order, never confirmed
cancelledOrder.Status = "Shipped";   // a Cancelled order, shipped
```

OrderModels/Domain, after Step 2

```
order.Status = OrderStatus.Shipped;    // no longer compiles
```

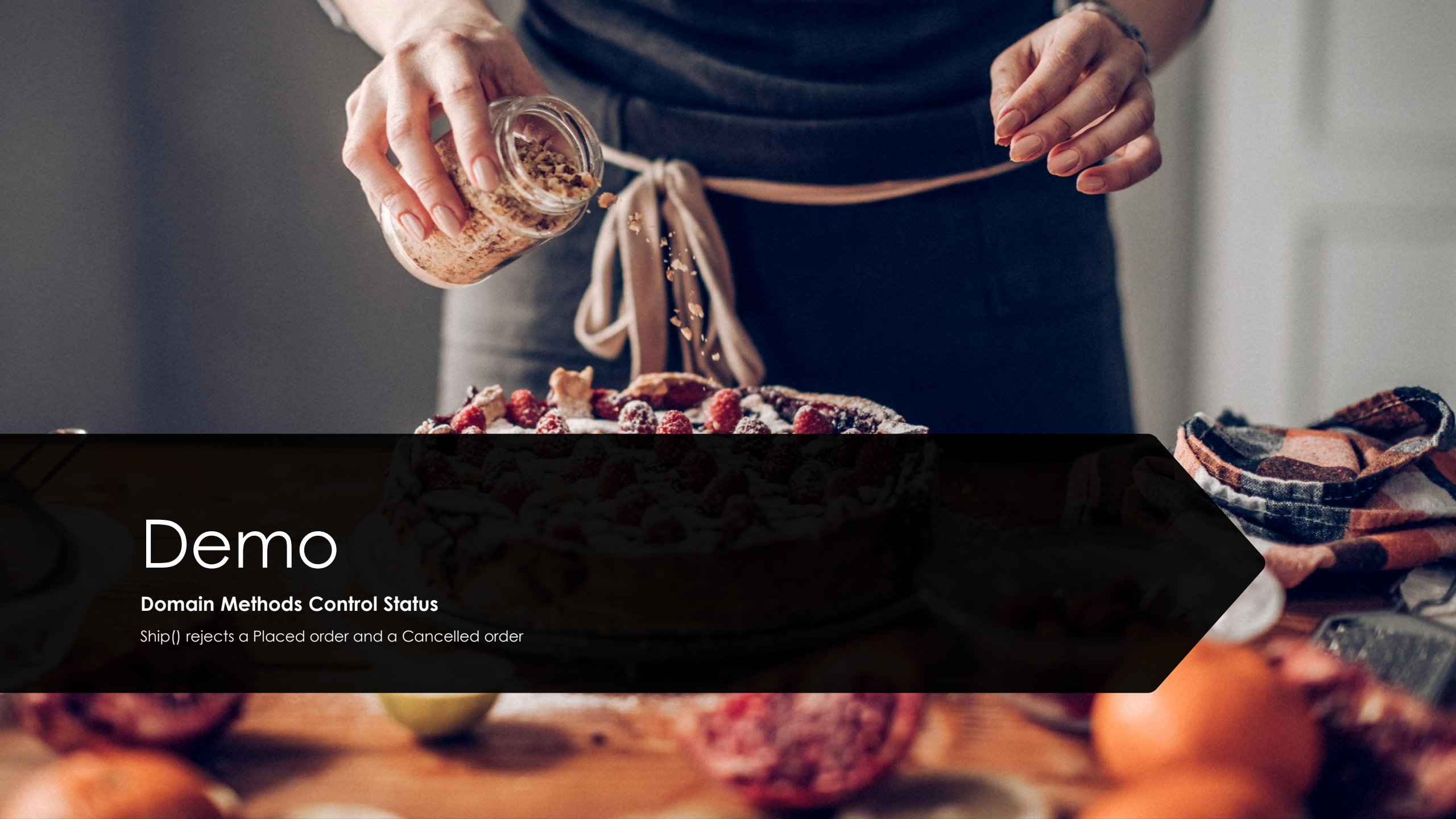
Nothing in the domain Order can move Status forward yet.

Domain Methods Guard Transitions

OrderModels/Domain/Order.cs, after Step 3

```
public void Ship()
{
    if (Status != OrderStatus.Confirmed)
        throw new InvalidOperationException($"Cannot ship an order in {Status} status.");
    Status = OrderStatus.Shipped;
}

public void Cancel()
{
    if (Status == OrderStatus.Shipped)
        throw new InvalidOperationException("Cannot cancel an order that has shipped.");
    Status = OrderStatus.Cancelled;
}
```



Demo

Domain Methods Control Status

Ship() rejects a Placed order and a Cancelled order



Total as a Money Value Object

Every total carries an amount and a currency

Value Object: The Problem

OrderModels/Anemic

```
public decimal Total { get; set; }

// Line 1: 19.99 USD    Line 2: 25.00 EUR
order.Total = order.Lines
    .Sum(orderLine => orderLine.LineTotal.Amount);
// Total: 44.99   (dollars? euros?)
```

A decimal Total has no currency.

Money Guards: The Problem

OrderModels/Anemic

```
Money.Create(-42.50m, "USD");                                    // -42.50 USD
Money.Create(19.99m, "USD").Add(Money.Create(25.00m, "EUR"));    // 44.99 USD
```

Nothing checks the amount or the currency, so every value is accepted.

Using a Value Object for the Total

OrderModels/Domain/Order.cs, after Step 4

```
public Money Total { get; private set; } = null!;

private void RecalculateTotal()
{
    Total = Lines
        .Select(orderLine => orderLine.LineTotal)
        .Aggregate((runningTotal, nextLineTotal) => runningTotal.Add(nextLineTotal));
}
```

Mapping Money with OwnsOne

TalkApp/Data/TalkDbContext.cs, after Step 4

```
orderBuilder.OwnsOne(order => order.Total, moneyBuilder =>
{
    moneyBuilder.Property(moneyValue => moneyValue.Amount)
        .HasColumnName("Total_Amount").HasPrecision(18, 2);
    moneyBuilder.Property(moneyValue => moneyValue.Currency)
        .HasColumnName("Total_Currency").HasMaxLength(10);
});
```

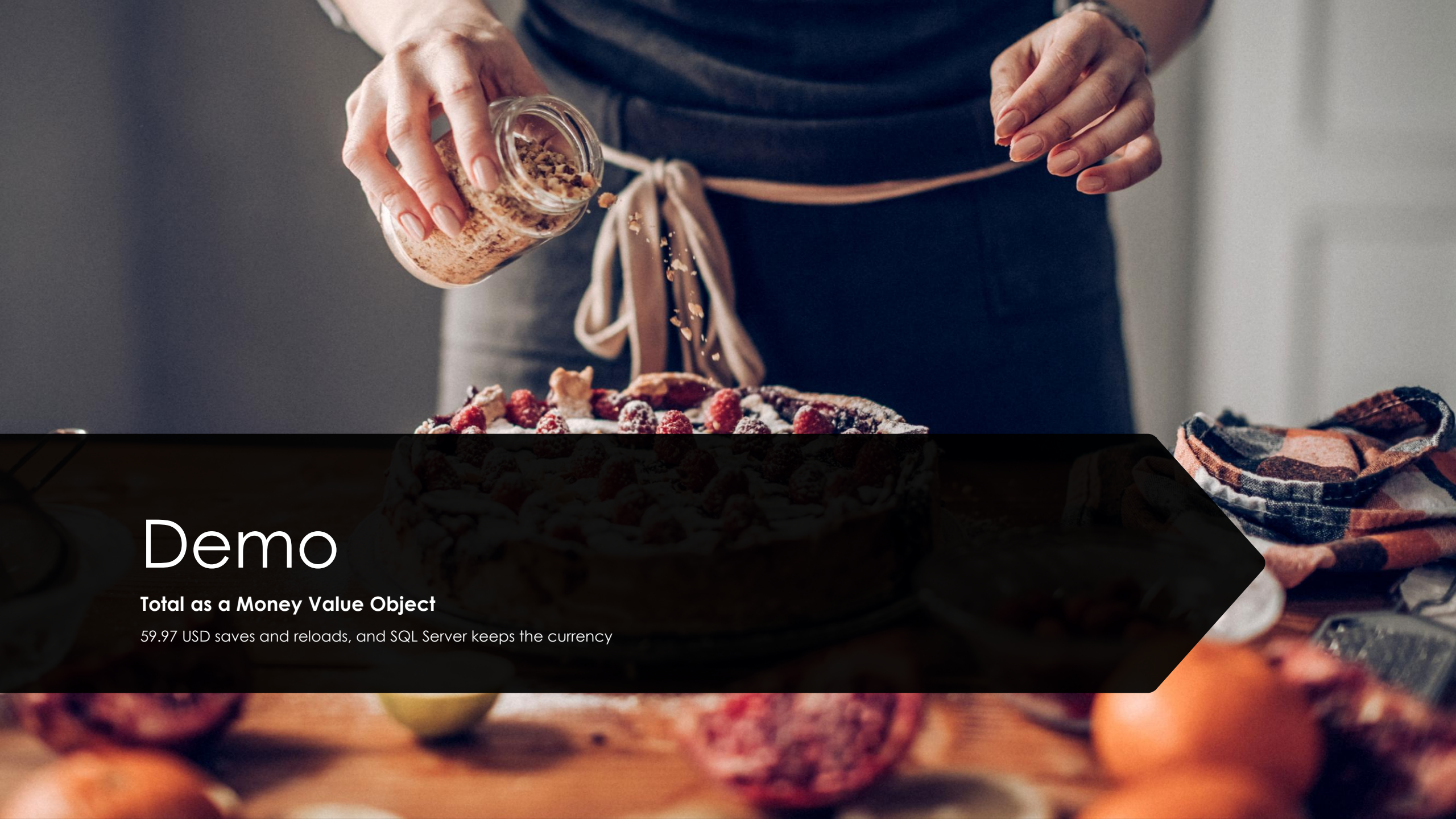
Total_Amount and Total_Currency are two columns on the DomainOrders table.

Guard Clauses in Money

OrderModels/Domain/Money.cs, after Step 4

```
// Money.Create
if (amount < 0)
    throw new ArgumentException("Amount cannot be negative.", nameof(amount));
if (string.IsNullOrWhiteSpace(currency))
    throw new ArgumentException("Currency cannot be empty.", nameof(currency));

// Money.Add
if (other.Currency != Currency)
    throw new InvalidOperationException(
        $"Cannot add {other.Currency} to {Currency}: currency mismatch.");
```



Demo

Total as a Money Value Object

59.97 USD saves and reloads, and SQL Server keeps the currency



Backing Field and Read-Only Lines

Every new line goes through AddLine, and Total stays correct

Backing Field: The Problem

OrderModels/Anemic

```
public List<OrderLine> Lines { get; set; } = new();

order.Lines.Add(new OrderLine(2002, 2, Money.Create(5.00m, "USD")));
// Lines: 2   Sum of lines: 20.00   Total: 10.00

order.Lines.Clear();
// Lines: 0   Total: 10.00
```

Any caller can change Lines, and nothing recalculates Total.

A Private Backing Field

OrderModels/Domain, after Step 5

```
private readonly List<OrderLine> _lines = new();
public IReadOnlyCollection<OrderLine> Lines => _lines.AsReadOnly();

public void AddLine(int productId, int quantity, Money unitPrice)
{
    _lines.Add(new OrderLine(productId, quantity, unitPrice));
    RecalculateTotal();
}

private void RecalculateTotal()
{
    Total = Lines
        .Select(orderLine => orderLine.LineTotal)
        .Aggregate((runningTotal, nextLineTotal) => runningTotal.Add(nextLineTotal));
}

// OrderLine.cs
internal OrderLine(int productId, int quantity, Money unitPrice)
```

Mapping the Backing Field

TalkApp/Data/TalkDbContext.cs, after Step 5

```
orderBuilder.Navigation(order => order.Lines)
    .HasField("_lines")
    .UsePropertyAccessMode(PropertyAccessMode.Field);
```

EF Core finds _lines by convention when the name matches Lines.
Any other field name requires HasField.

Line Guards: The Problem

OrderModels/Anemic

```
order.Lines.Add(new OrderLine(2001, -3, Money.Create(19.99m, "USD")));
order.Lines.Add(new OrderLine(0, 1, Money.Create(5.00m, "USD")));
order.Lines.Add(new OrderLine(2001, 2, Money.Create(19.99m, "USD")));
```

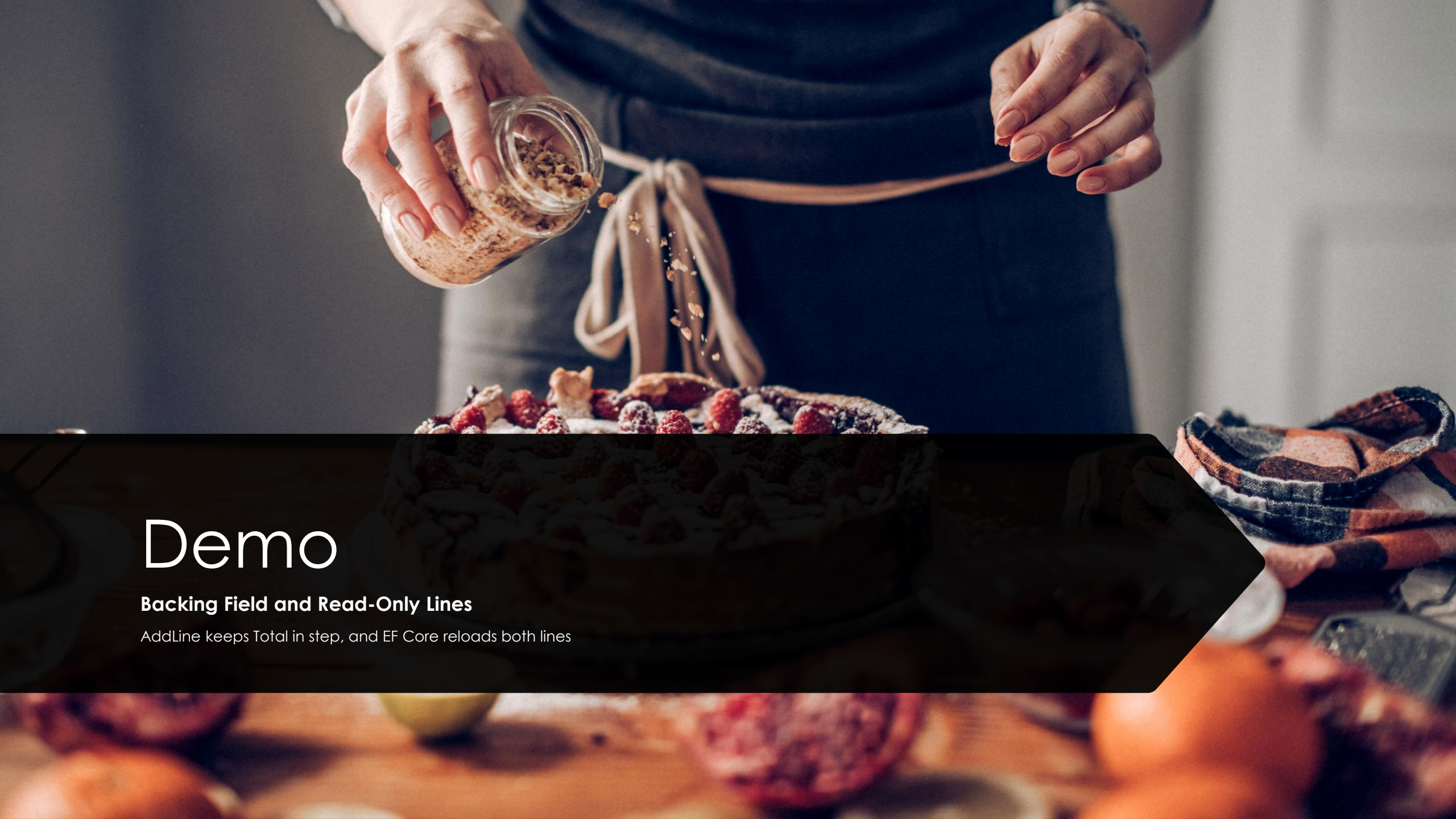
Nothing checks the lines, so every line is accepted.

Guard Clauses on the Lines

OrderModels/Domain, after Step 5

```
// Order.AddLine
if (_lines.Any(orderLine => orderLine.ProductId == productId))
    throw new InvalidOperationException($"Product {productId} is already on this order.");

// OrderLine constructor
if (productId <= 0)
    throw new ArgumentException("A line needs a valid product id.", nameof(productId));
if (quantity <= 0)
    throw new ArgumentException("Quantity must be greater than zero.", nameof(quantity));
```



Demo

Backing Field and Read-Only Lines

AddLine keeps Total in step, and EF Core reloads both lines



Seven Techniques in Review

What each technique prevents, and both Order classes compared

Seven Techniques, Seven Rules

Step	Technique	What it prevents	Code that enforces it
1	Private constructor	An order that skips Order.Place	private Order(), EF Core materialization
1	Factory method	Orders built with missing or invalid values	Order.Place
2	Private setters	Callers rewriting a placed order	private set on six properties
3	Domain methods	Shipping an unconfirmed or cancelled order	Process, Confirm, Ship, Cancel
4	Money value object	A total with no currency	Money, RecalculateTotal, OwnsOne
5	Read-only backing field	A total that differs from the sum of the lines	_lines, AddLine, RecalculateTotal, HasField
1, 4, 5	Guard clauses	Bad input, duplicates, mixed currencies	Guards in Place, AddLine, OrderLine, Money

The Anemic and Domain Orders

OrderModels/Anemic/Order.cs

```
public int Id { get; set; }
public string Status { get; set; } = string.Empty;
public decimal Total { get; set; }
public DateTime OrderDate { get; set; }
public string CustomerName { get; set; } = string.Empty;
public List<OrderLine> Lines { get; set; } = new();
```

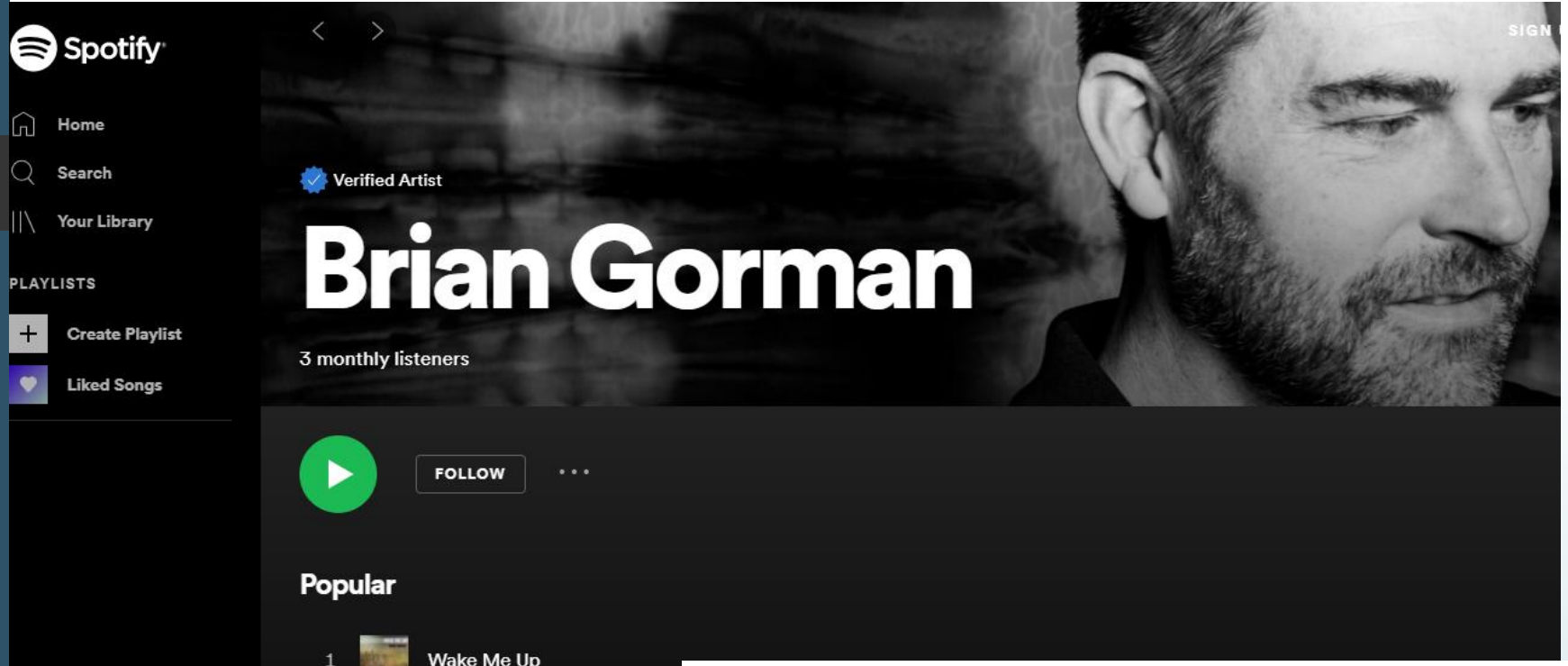
OrderModels/Domain/Order.cs, after Step 5

```
public int Id { get; private set; }
public OrderStatus Status { get; private set; }
public Money Total { get; private set; } = null!;
public DateTime OrderDate { get; private set; }
public string CustomerName { get; private set; } = string.Empty;
private readonly List<OrderLine> _lines = new();
public IReadOnlyCollection<OrderLine> Lines => _lines.AsReadOnly();
private Order() { }
```



Questions?

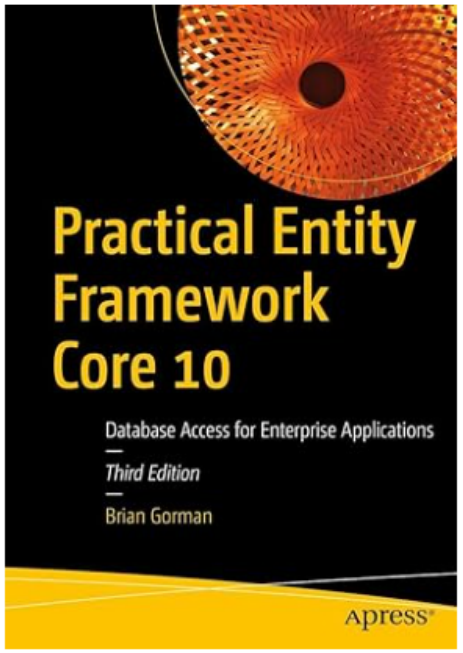
- Email:
  - brian@majorguidancesolutions.com
  - blgorman@gmail.com
- Twitter
  - @blgorman
- Linked In
  - https://www.linkedin/in/brianlgorman



@blgorman

https://www.linkedin.com/in/brianlgorman

Thanks for having me today!



Practical Entity Framework Core 10: Database Access for Enterprise Applications Third Edition

by Brian Gorman (Author)

Savings Pre-order Price Guarantee. Terms

Focus on the code-first approach to developing a database with EFCore 10. Taking the code-first approach allows the entire database to be built and maintained in code so there is never a situation in which you cannot restore the database schema. Additionally, the code-first approach creates an entirely transparent record of changes to the database that is easily tracked in your source control. Emphasis throughout the book is on leaving you well positioned to architect and lead data development efforts for your organization.

New to This Edition

Demonstrate the Object Relational Mapper (ORM) of choice for working with databases for enterprise applications using .Net Framework from the ground up. This third edition is revised to cover modern EFCore development as of EFCore 10 and includes information and examples for changes from EFCore 7, 8, 9, and 10. Throughout the book, practical examples and considerations are presented to help you understand how to create robust and efficient database solutions.

What You Will Learn

Read more